

Implement Snowflake Streams for Sales Data Processing

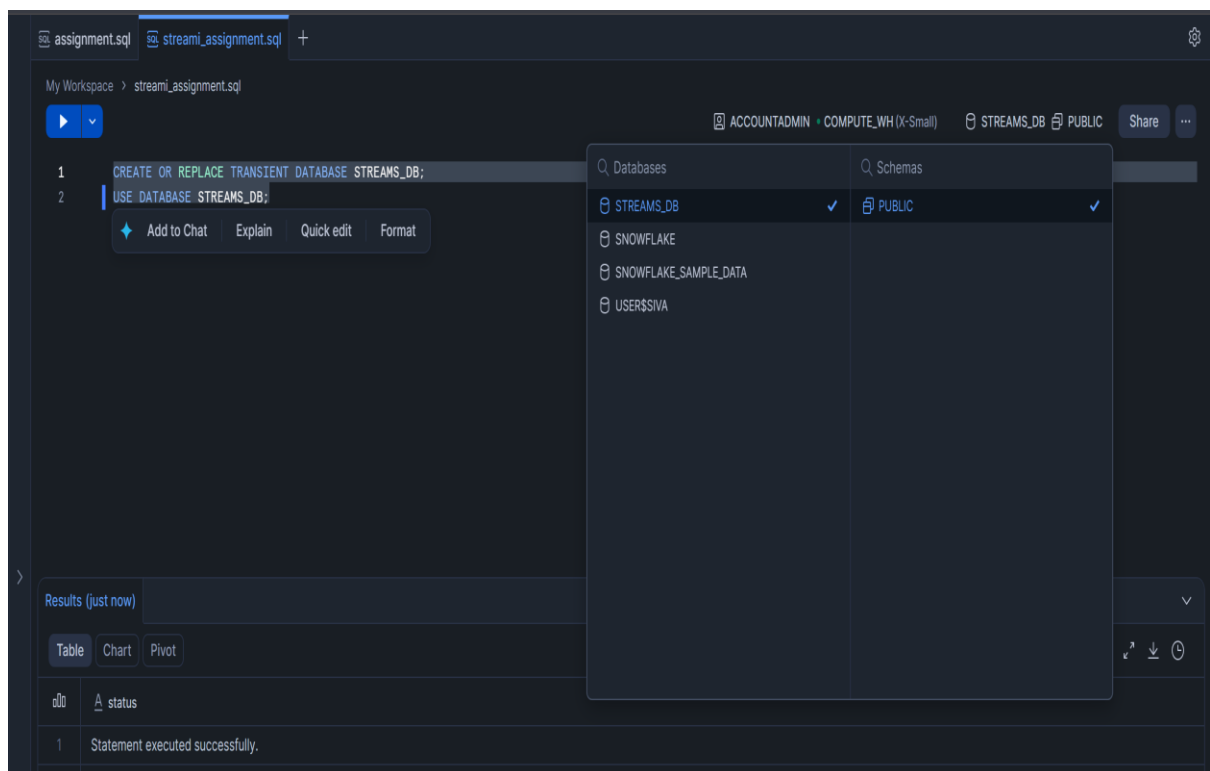
A retail company receives sales data in a staging table. The company wants to load this data into a final reporting table. After the initial load, only newly inserted or updated records should be processed using Snowflake Streams.

You need to create the database, tables, stream object, insert records, consume stream data, and update the final table using stream changes.

Task 1: Create Database

Create a transient database named:

STREAMS_DB



Task 2: Create Raw Sales Staging Table

Create a table named:

SALES_RAW_STAGING

The table should contain the following columns:

id

product

price

amount

store_id

Use suitable data types.

The screenshot shows a SQL IDE interface with a dark theme. The top bar displays the file name 'streams_assignment.sql' and a plus icon. Below the top bar, the workspace path 'My Workspace > streams_assignment.sql' is shown. The main editor area contains the following SQL code:

```
4
5 CREATE OR REPLACE TABLE STREAMS_DB.PUBLIC.SALES_RAW_STAGING (
6     id INT,
7     product STRING,
8     price FLOAT,
9     amount INT,
10    store_id INT
11 );
12
13 select * from STREAMS_DB.PUBLIC.SALES_RAW_STAGING;
```

The bottom panel shows the 'Results (just now)' section. It includes tabs for 'Table', 'Chart', and 'Pivot'. The 'Table' tab is selected, showing a table with 5 columns: ID, PRODUCT, PRICE, AMOUNT, and STORE_ID. The table is currently empty, with 0 rows displayed. The execution time is 35ms.

ID	PRODUCT	PRICE	AMOUNT	STORE_ID
----	---------	-------	--------	----------

Task 3: Insert Initial Sales Data

Insert the following five records into SALES_RAW_STAGING:

id	product	price	amount	store_id
1	Banana	1.99	1	1
2	Lemon	0.99	1	1
3	Apple	1.79	1	2
4	Orange Juice	1.89	1	1
5	Cereals	5.98	2	1

```
12
13 select * from STREAMS_DB.PUBLIC.SALES_RAW_STAGING;
14
15 INSERT INTO SALES_RAW_STAGING VALUES
16 (1, 'Banana', 1.99, 1, 1),
17 (2, 'Lemon', 0.99, 1, 1),
18 (3, 'Apple', 1.79, 1, 2),
19 (4, 'Orange Juice', 1.89, 1, NULL),
20 (5, 'Cereals', 5.98, 2, 1);
```

Results (just now)

Table Chart Pivot 5 rows 92ms

# ID	# PRICE	# AMOUNT	# STORE_ID
1	1.99	1	1
2	0.99	1	1
3	1.79	1	2
4	1.89	1	null
5	5.98	2	1

Task 4: Create Store Table

Create a table named:

STORE_TABLE

The table should contain:

store_id

location

employees

```
22
23 CREATE OR REPLACE TABLE STREAMS_DB.PUBLIC.STORE_TABLE (
24     store_id INT,
25     location STRING,
26     employees INT
27 );
28 select * from STREAMS_DB.PUBLIC.STORE_TABLE;
```

Results (just now)

Table Chart Pivot 0 rows 26ms

	STORE_ID	LOCATION	EMPLOYEES
--	----------	----------	-----------

Task 5: Insert Store Data

Insert the following records into STORE_TABLE:

store_id locationemployees

1 Chicago33

2 London 12

```
27
28 select * from STREAMS_DB.PUBLIC.STORE_TABLE;
29
30 INSERT INTO STREAMS_DB.PUBLIC.STORE_TABLE VALUES
31 (1,'Chicago',33),
32 (2,'London',12);
```

Results (just now)

Table Chart Pivot 2 rows 72ms

#	STORE_ID	LOCATION	EMPLOYEES
1	1	Chicago	33
2	2	London	12

Task 6: Create Final Sales Table

Create a table named:

SALES_FINAL_TABLE

The table should contain:

id

product

price

amount

store_id

location

employees

```
34 CREATE OR REPLACE TABLE STREAMS_DB.PUBLIC.SALES_FINAL_TABLE (  
35     id INT,  
36     product STRING,  
37     price FLOAT,  
38     amount INT,  
39     store_id INT,  
40     location STRING,  
41     employees INT  
42 );  
43  
44 select * from STREAMS_DB.PUBLIC.SALES_FINAL_TABLE;
```

Results (just now) ▼

Table Chart Pivot 0 rows 31ms 🔍 📄 📥 🕒

ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
----	---------	-------	--------	----------	----------	-----------

This table will store sales data along with store location and employee details.

Task 7: Perform Initial Full Load

Load data from SALES_RAW_STAGING into SALES_FINAL_TABLE.

While loading, join:

SALES_RAW_STAGING

with:

STORE_TABLE

using STORE_ID.

```
45
46 INSERT INTO SALES_FINAL_TABLE
47 SELECT
48     S.id,
49     S.product,
50     S.price,
51     S.amount,
52     S.store_id,
53     ST.location,
54     ST.employees
55 FROM SALES_RAW_STAGING S
56 LEFT JOIN STORE_TABLE ST
57 ON S.store_id = ST.store_id;
```

Results (just now) ▼

Table Chart Pivot

Q [7] 1 row ⓘ 251ms 🔍 ⬇ ⌚

id	# number of rows inserted
1	5

Task 8: Verify Initial Load

Display data from:

SALES_RAW_STAGING;

55

56

57

58

59

SELECT * FROM STREAMS_DB.PUBLIC.SALES_RAW_STAGING;

SELECT * FROM STREAMS_DB.PUBLIC.STORE_TABLE;

SELECT * FROM STREAMS_DB.PUBLIC.SALES_FINAL_TABLE;

Add to Chat

Explain

Quick edit

Format

Results (just now)

TableChartPivot

5 rows50ms

# ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	Banana	1.99	1	1
2	Lemon	0.99	1	1
3	Apple	1.79	1	2
4	Orange Juice	1.89	1	null
5	Cereals	5.98	2	1

STORE_TABLE;

6

7

8

9

SELECT * FROM STREAMS_DB.PUBLIC.STORE_TABLE;

SELECT * FROM STREAMS_DB.PUBLIC.SALES_FINAL_TABLE;

Add to Chat

Explain

Quick edit

Format

Results (just now)

TableChartPivot

2 rows46ms

STORE_ID	LOCATION	EMPLOYEES
1	Chicago	33
2	London	12

SALES_FINAL_TABLE;

55

56

57

58

SELECT * FROM STREAMS_DB.PUBLIC.SALES_FINAL_TABLE;

Add to Chat

Explain

Quick edit

Format

Results (just now)

TableChartPivot

5 rows25ms

# ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	Banana	1.99	1	1	Chicago	33
2	Lemon	0.99	1	1	Chicago	33
3	Apple	1.79	1	2	London	12
4	Orange Juice	1.89	1	null	null	null
5	Cereals	5.98	2	1	Chicago	33

Confirm that SALES_FINAL_TABLE contains 5 records.

Stream Implementation

Task 9: Create Stream Object

Create a stream named:

`SALES_STREAM`

on top of:

`SALES_RAW_STAGING`

```
60
61 CREATE OR REPLACE STREAM SALES_STREAM
62 ON TABLE SALES_RAW_STAGING;
63
64
```

Results (just now)

Table Chart Pivot 1 row 192ms

1	Stream SALES_STREAM successfully created.
---	---

Task 10: View Stream Information

Execute commands to:

Show all streams.

```
63
64 SHOW STREAMS;
65
66
```

Results (just now)

Table Chart Pivot 1 row 33ms

created_on	name	database_name	schema_name	owner	comment	table_name	source_type	base_tables
2026-05-18 04:27:44	SALES_STREAM	STREAMS_DB	PUBLIC	ACCOUNTADMIN		STREAMS_DB.PUBLIC.	Table	STREAMS_DB.PUBLIC.

Describe the `SALES_STREAM`.

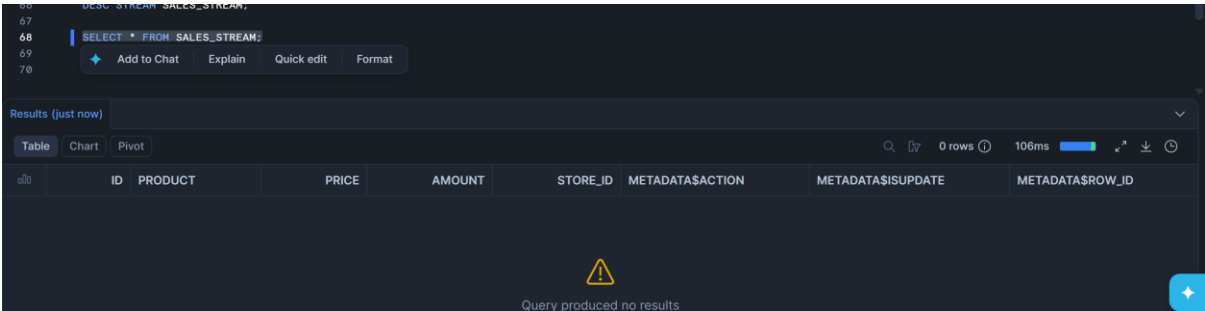
```
DESC STREAM SALES_STREAM;
```

Results (just now)

Table Chart Pivot 1 row 34ms

created_on	name	database_name	schema_name	owner	comment	table_name	source_type	base_tables
2026-05-18 04:27:44	SALES_STREAM	STREAMS_DB	PUBLIC	ACCOUNTADMIN		STREAMS_DB.PUBLIC.	Table	STREAMS_DB.PUBLIC.

Select data from SALES_STREAM.



The screenshot shows a SQL query editor with the query `SELECT * FROM SALES_STREAM;`. Below the query, the results section is titled "Results (just now)" and shows a table with columns: ID, PRODUCT, PRICE, AMOUNT, STORE_ID, METADATA\$ACTION, METADATA\$ISUPDATE, and METADATA\$ROW_ID. The table is empty, and a message at the bottom states "Query produced no results".

ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
----	---------	-------	--------	----------	------------------	--------------------	------------------

Answer this question:

Why is SALES_STREAM empty immediately after creation?

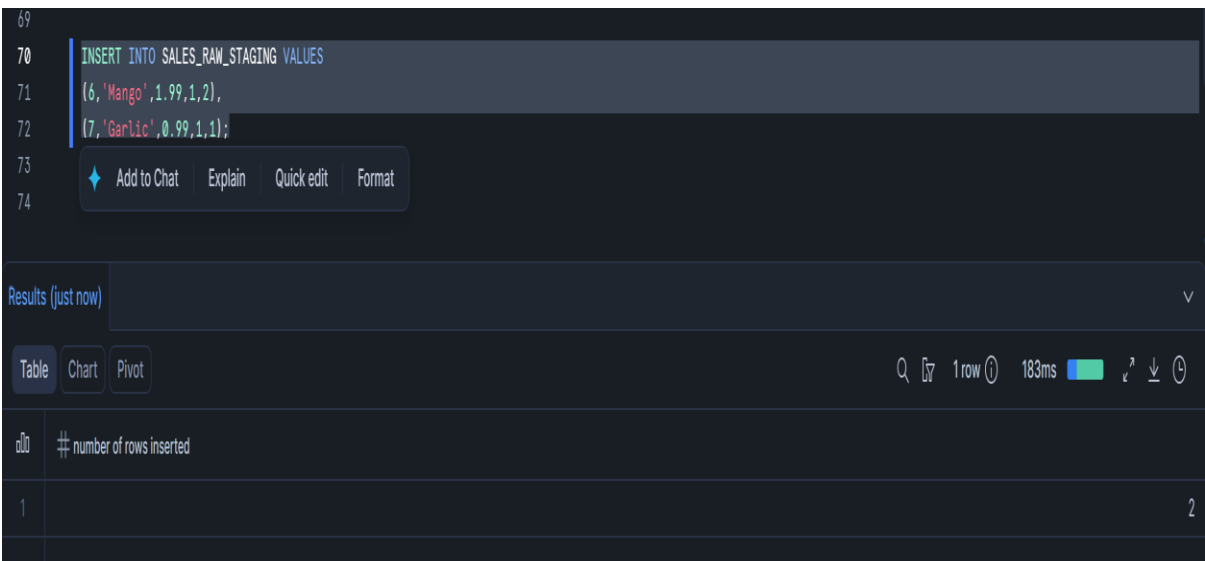
BECAUSE STREAM CAPTURES ONLY NEW CHANGES AFTER CREATION

Insert Change Capture

Task 11: Insert New Sales Records

Insert the following two records into SALES_RAW_STAGING:

id	product	price	amount	store_id
6	Mango	1.99	1	2
7	Garlic	0.99	1	1



The screenshot shows a SQL query editor with the query `INSERT INTO SALES_RAW_STAGING VALUES (6, 'Mango', 1.99, 1, 2), (7, 'Garlic', 0.99, 1, 1);`. Below the query, the results section is titled "Results (just now)" and shows a table with columns: # number of rows inserted. The table has one row with the value 2.

number of rows inserted
2

Task 12: Check Stream Data

70

71

72

73

74

75

76

```
INSERT INTO SALES_RAW_STAGING VALUES
(6, 'Mango', 1.99, 1, 2),
(7, 'Garlic', 0.99, 1, 1);
```

```
SELECT * FROM SALES_STREAM;
```

Add to Chat

Explain

Quick edit

Format

Results (just now)

TableChartPivot

Q 2 rows 119ms

# ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
1	Mango	1.99	1	2	INSERT	FALSE	6b5670d0e2e3efe69dcfbba5c049adbd1666cf1
2	Garlic	0.99	1	1	INSERT	FALSE	d339500f0570498304767f5fa9d1530910e7ce06

Query SALES_STREAM.

Observe the new records and metadata columns:

METADATA\$ACTION

METADATA\$ISUPDATE

METADATA\$ROW_ID

Answer this question:

What value do you see in METADATA\$ACTION for newly inserted records?

INSERT

Task 13: Verify Source and Target Tables

Check data in:

SALES_RAW_STAGING;

75

76

77

78

```
SELECT * FROM SALES_RAW_STAGING;
```

Add to Chat

Explain

Quick edit

Format

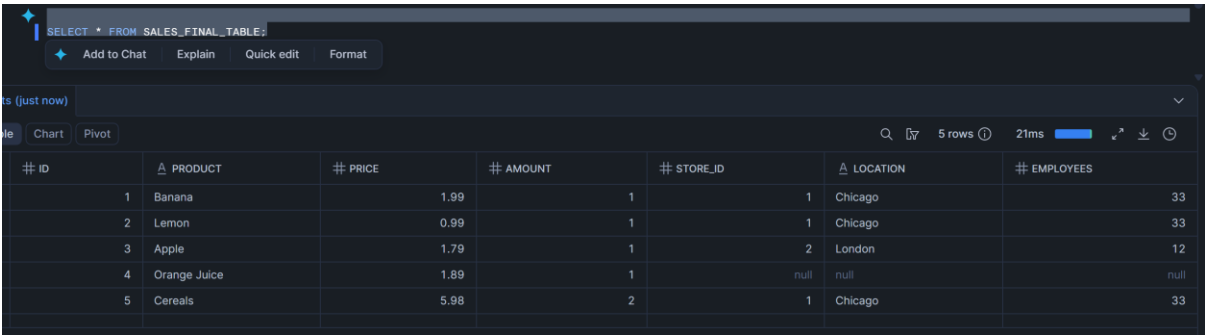
Results (just now)

TableChartPivot

Q 7 rows 62ms

# ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	Banana	1.99	1	1
2	Lemon	0.99	1	1
3	Apple	1.79	1	2
4	Orange Juice	1.89	1	null
5	Cereals	5.98	2	1
6	Mango	1.99	1	2
7	Garlic	0.99	1	1

SALES_FINAL_TABLE;



The screenshot shows a SQL query editor with a dark theme. The query bar contains the text "SELECT * FROM SALES_FINAL_TABLE;". Below the query bar are buttons for "Add to Chat", "Explain", "Quick edit", and "Format". The results section shows a table with 5 rows and 7 columns. The columns are labeled: # ID, PRODUCT, PRICE, AMOUNT, STORE_ID, LOCATION, and EMPLOYEES. The data rows are as follows:

# ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	Banana	1.99	1	1	Chicago	33
2	Lemon	0.99	1	1	Chicago	33
3	Apple	1.79	1	2	London	12
4	Orange Juice	1.89	1	null	null	null
5	Cereals	5.98	2	1	Chicago	33

Answer this question:

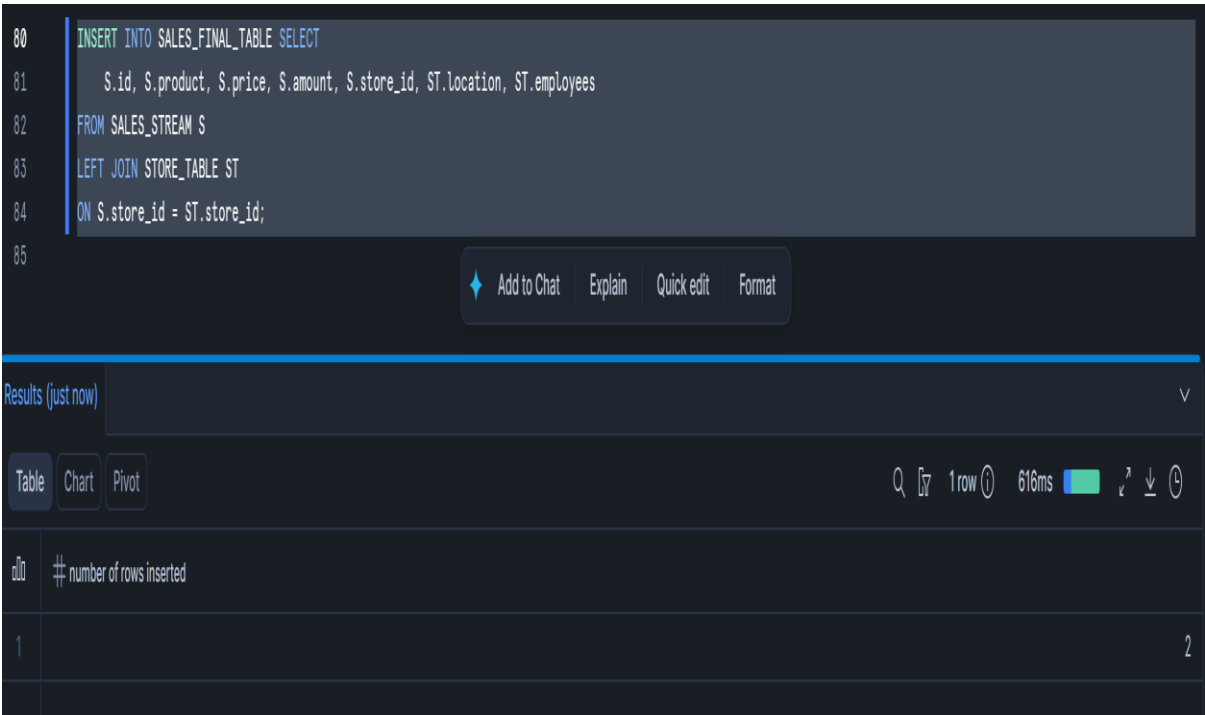
Are the new records available in SALES_FINAL_TABLE before consuming the stream?

NO NEW RECORDS ARE NOT FOUND IN FINAL TABLE

Task 14: Consume Stream Data

Insert new records from SALES_STREAM into SALES_FINAL_TABLE.

While inserting, join the stream with STORE_TABLE using STORE_ID.



The screenshot shows a SQL query editor with a dark theme. The query bar contains the text:

```
80 INSERT INTO SALES_FINAL_TABLE SELECT
81     S.id, S.product, S.price, S.amount, S.store_id, ST.location, ST.employees
82 FROM SALES_STREAM S
83 LEFT JOIN STORE_TABLE ST
84 ON S.store_id = ST.store_id;
85
```

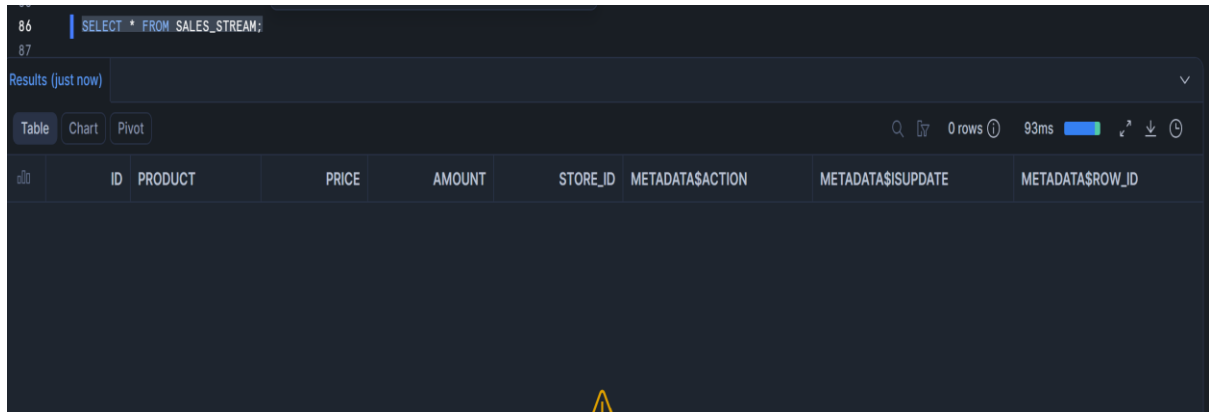
Below the query bar are buttons for "Add to Chat", "Explain", "Quick edit", and "Format". The results section shows a table with 1 row and 1 column. The column is labeled: # number of rows inserted. The data row is as follows:

number of rows inserted
2

Task 15: Verify Stream After Consumption

Query:

```
SELECT * FROM SALES_STREAM;
```



The screenshot shows a SQL query interface with the query `SELECT * FROM SALES_STREAM;` entered. Below the query, the results are displayed in a table format. The table has columns: ID, PRODUCT, PRICE, AMOUNT, STORE_ID, METADATA\$ACTION, METADATA\$ISUPDATE, and METADATA\$ROW_ID. The results section shows "0 rows" and a execution time of "93ms".

ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
----	---------	-------	--------	----------	------------------	--------------------	------------------

Answer this question:

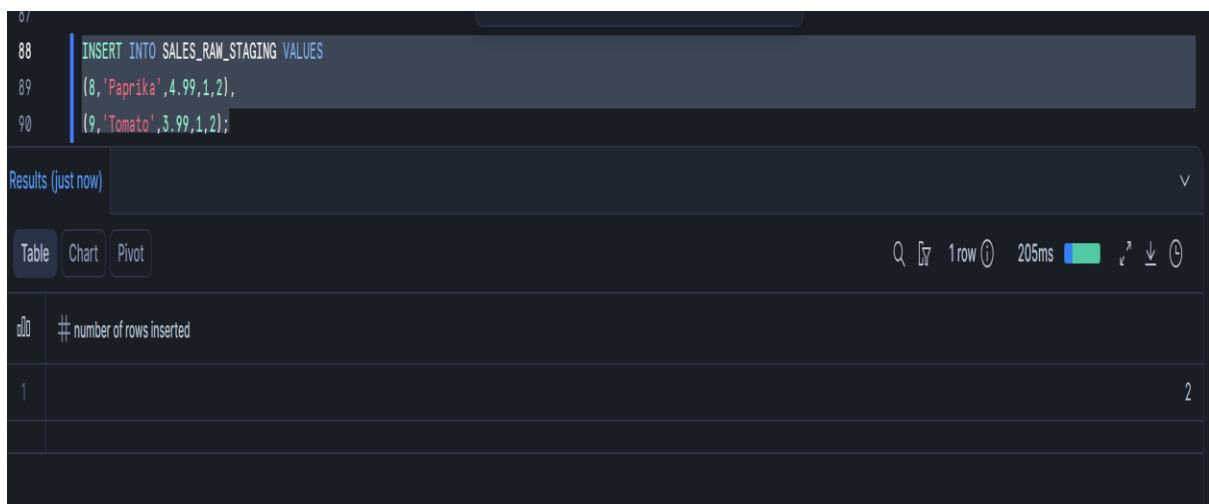
Why did the stream become empty after inserting data into SALES_FINAL_TABLE?

STREAM BECOMES EMPTY BECAUSE DATA IS CONSUMED

Task 16: Insert More Sales Records

Insert the following two records into SALES_RAW_STAGING:

id	product	price	amount	store_id
8	Paprika	4.99	1	2
9	Tomato	3.99	1	2



The screenshot shows a SQL query interface with the query `INSERT INTO SALES_RAW_STAGING VALUES (8, 'Paprika', 4.99, 1, 2), (9, 'Tomato', 3.99, 1, 2);` entered. Below the query, the results are displayed in a table format. The table has columns: # number of rows inserted. The results section shows "1 row" and a execution time of "205ms".

number of rows inserted
1

Task 17: Consume Second Stream Data

Consume the newly captured stream records and insert them into SALES_FINAL_TABLE.

92 INSERT INTO SALES_FINAL_TABLE SELECT
93 S.id, S.product, S.price, S.amount, S.store_id, ST.location, ST.employees
94 FROM SALES_STREAM S
95 LEFT JOIN STORE_TABLE ST
96 ON S.store_id = ST.store_id;
97

Add to Chat Explain Quick edit Format

Results (just now)

Table Chart Pivot

1 row 349ms

#	# number of rows inserted
1	2

Task 18: Verify Final Insert Output

Display data from:

SALES_FINAL_TABLE;

SALES_RAW_STAGING;

SALES_STREAM;

97
98 SELECT * FROM SALES_FINAL_TABLE;

Add to Chat Explain Quick edit Format

Results (just now)

Table Chart Pivot

9 rows 69ms

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	Banana	1.99	1	1	Chicago	33
2	2	Lemon	0.99	1	1	Chicago	33
3	3	Apple	1.79	1	2	London	12
4	4	Orange Juice	1.89	1	null	null	null
5	5	Cereals	5.98	2	1	Chicago	33
6	7	Garlic	0.99	1	1	Chicago	33
7	6	Mango	1.99	1	2	London	12
8	8	Paprika	4.99	1	2	London	12
9	9	Tomato	3.99	1	2	London	12

Answer:

How many records are now available in SALES_FINAL_TABLE? 9

Update Change Capture

Task 19: Check Existing Data Before Update

Display records from:

SALES_RAW_STAGING;

99
100
101
102
103

SELECT * FROM SALES_RAW_STAGING;

Add to Chat

Explain

Quick edit

Format

Results (just now)

TableChartPivot

9 rows57ms

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	Banana	1.99	1	1
2	2	Lemon	0.99	1	1
3	3	Apple	1.79	1	2
4	4	Orange Juice	1.89	1	null
5	5	Cereals	5.98	2	1
6	6	Mango	1.99	1	2
7	7	Garlic	0.99	1	1
8	8	Paprika	4.99	1	2
9	9	Tomato	3.99	1	2

SALES_STREAM;

101
102

SELECT * FROM SALES_STREAM;

Add to Chat

Explain

Quick edit

Format

Results (just now)

TableChartPivot

0 rows88ms

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ISACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
---	----	---------	-------	--------	----------	--------------------	--------------------	------------------

Task 20: Perform First Update

Update the product name:

Banana to: Potato

in SALES_RAW_STAGING.

103
104
105
106

UPDATE SALES_RAW_STAGING
SET product = 'Potato'
WHERE product = 'Banana';

Add to Chat

Explain

Quick edit

Format

Results (just now)

TableChartPivot

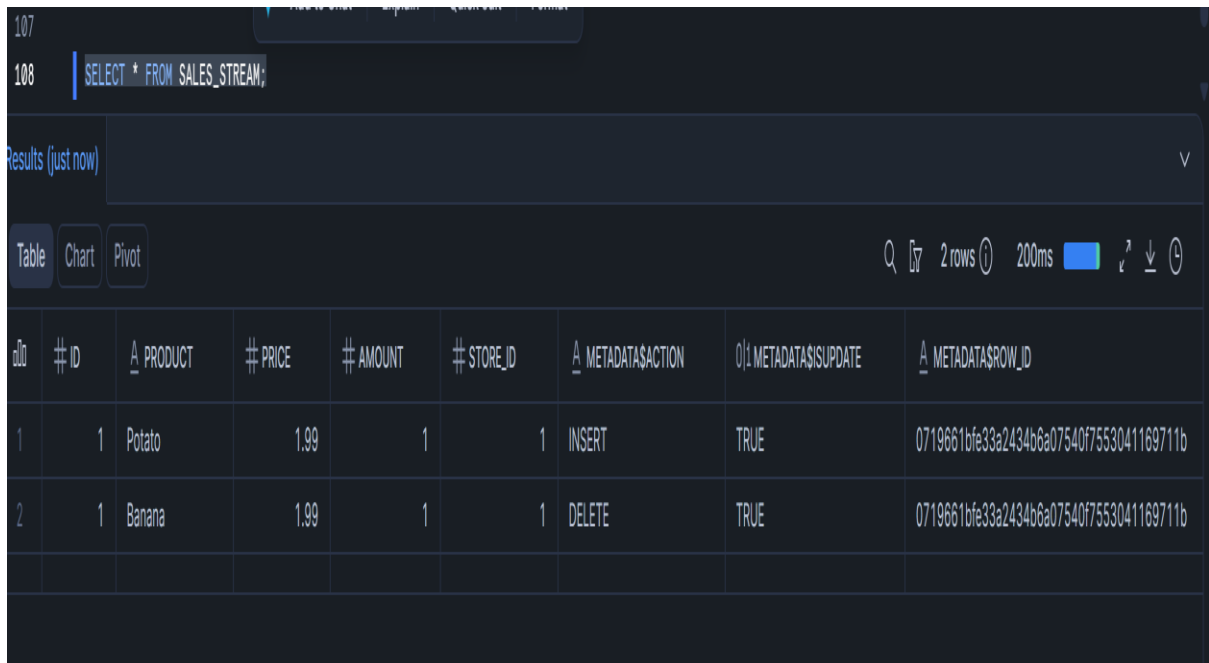
1 row234ms

#	number of rows updated	number of multi-joined rows updated
1	1	0

Task 21: Check Stream After Update

Query:

```
SELECT * FROM SALES_STREAM;
```



The screenshot shows a SQL query execution interface. The query entered is `SELECT * FROM SALES_STREAM;`. The results are displayed in a table with 9 columns: `ID`, `PRODUCT`, `PRICE`, `AMOUNT`, `STORE_ID`, `METADATA$ACTION`, `METADATA$ISUPDATE`, and `METADATA$ROW_ID`. There are 2 rows of data. The first row shows an INSERT action for a Potato with a price of 1.99 and an amount of 1. The second row shows a DELETE action for a Banana with a price of 1.99 and an amount of 1. The interface also shows a 'Results (just now)' label, a 'Table' button, and a '2 rows' indicator.

ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
1	Potato	1.99	1	1	INSERT	TRUE	0719661bfe33a2434b6a07540f7553041169711b
2	Banana	1.99	1	1	DELETE	TRUE	0719661bfe33a2434b6a07540f7553041169711b

Answer:

How does Snowflake represent an UPDATE operation inside a stream?

INSERT

DELETE

Task 22: Consume Update Using MERGE

Use a MERGE statement to update the corresponding record in SALES_FINAL_TABLE.

Condition:

Match records using ID

Update only records where `METADATA$ACTION = 'INSERT'`

and `METADATA$ISUPDATE = TRUE`

Update the following columns:

product

price

amount

store_id

```
109
110 MERGE INTO SALES_FINAL_TABLE T
111 USING SALES_STREAM S
112 ON T.id = S.id
113 WHEN MATCHED
114 AND S.METADATA$ACTION = 'INSERT'
115 AND S.METADATA$ISUPDATE = TRUE
116 THEN UPDATE
117 SET T.product = S.product, T.price = S.price, T.amount = S.amount, T.store_id = S.store_id;
```

Results (just now) ▼

Table Chart Pivot Q [?] 1 row 378ms

1	1
---	---

Task 23: Verify First Update

Display data from:

SALES_FINAL_TABLE;

SALES_RAW_STAGING;

SALES_STREAM;

118

119 `SELECT * FROM SALES_FINAL_TABLE;`

Results (just now)

Table Chart Pivot

9 rows 967ms

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	Potato	1.99	1	1	Chicago	33
2	2	Lemon	0.99	1	1	Chicago	33
3	3	Apple	1.79	1	2	London	12
4	4	Orange Juice	1.89	1	null	null	null
5	5	Cereals	5.98	2	1	Chicago	33
6	7	Garlic	0.99	1	1	Chicago	33
7	6	Mango	1.99	1	2	London	12
8	8	Paprika	4.99	1	2	London	12
9	9	Tomato	3.99	1	2	London	12

Answer:

Is Banana changed to Potato in SALES_FINAL_TABLE?

YSE-UPDATED

Second Update Change Capture

Task 24: Perform Second Update

Update the product name:

Apple

to:

Green apple

in SALES_RAW_STAGING.

```

123 UPDATE SALES_RAW_STAGING
124 SET product = 'Green apple'
125 WHERE product = 'Apple';

```

Results (just now)

Table Chart Pivot 1 row 524ms

	# number of rows updated	# number of multi-joined rows updated
1	1	0

Task 25: Consume Second Update Using MERGE

Use a MERGE statement to update the changed record in SALES_FINAL_TABLE.

```

109
110 MERGE INTO SALES_FINAL_TABLE T
111 USING SALES_STREAM S
112 ON T.id = S.id
113 WHEN MATCHED
114 AND S.METADATA$ACTION = 'INSERT'
115 AND S.METADATA$ISUPDATE = TRUE
116 THEN UPDATE
117 SET T.product = S.product, T.price = S.price, T.amount = S.amount, T.store_id = S.store_id;
118

```

Results (just now)

Table Chart Pivot 1 row 404ms

	# number of rows updated
1	1

Task 26: Verify Final Output

Display data from:

SALES_FINAL_TABLE;

129
130 ✨ Ctrl+I to generate
131 | SELECT * FROM SALES_FINAL_TABLE;

Results (just now)

Table Chart Pivot

Q 9 rows 23ms

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	LOCATION	EMPLOYEES
1	1	Potato	1.99	1	1	Chicago	33
2	2	Lemon	0.99	1	1	Chicago	33
3	3	Green apple	1.79	1	2	London	12
4	4	Orange Juice	1.89	1	null	null	null
5	5	Cereals	5.98	2	1	Chicago	33
6	7	Garlic	0.99	1	1	Chicago	33
7	6	Mango	1.99	1	2	London	12
8	8	Paprika	4.99	1	2	London	12
9	9	Tomato	3.99	1	2	London	12

SALES_RAW_STAGING;

130
131 | SELECT * FROM SALES_RAW_STAGING;

Results (just now)

Table Chart Pivot

Q 9 rows 51ms

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID
1	1	Potato	1.99	1	1
2	2	Lemon	0.99	1	1
3	3	Green apple	1.79	1	2
4	4	Orange Juice	1.89	1	null
5	5	Cereals	5.98	2	1
6	6	Mango	1.99	1	2
7	7	Garlic	0.99	1	1
8	8	Paprika	4.99	1	2
9	9	Tomato	3.99	1	2

SALES_STREAM;

132
133 | SELECT * FROM SALES_STREAM;

Results (7 minutes ago)

Table Chart Pivot

Q 0 rows 73ms

#	ID	PRODUCT	PRICE	AMOUNT	STORE_ID	METADATA\$ACTION	METADATA\$ISUPDATE	METADATA\$ROW_ID
---	----	---------	-------	--------	----------	------------------	--------------------	------------------

Answer:

Is Apple changed to Green apple in SALES_FINAL_TABLE?

YES-UPDATED